



汇编语言

7 位操作类指令与数据处理类指令应用

张青

ieqzhang@zzu.edu.cn

教学内容



- 复习算术运算类指令

教材章节：4.3

慕课：5.4

- ▶ 状态标志
- ▶ 算术运算指令

- 学习位操作类指令

教材章节：4.4

慕课：6.2，6.3，6.4

- ▶ 位操作指令

- 学习数据处理类应用程序

教材章节：4.5

慕课：5.5，5.6，5.7，6.8，
6.9，6.10，6.11

- ▶ 自然数求和，处理器识别
温度转换
- ▶ C语言的运算表达式，截取
m位数据，查看状态标志

学习目标



- 能正确书写通用数据处理类指令
- 能判断指令执行结果及对标志位的影响
- 能编写指令序列完成简单的应用
- 能写出C语言程序对应的汇编指令
- 能写出汇编指令序列对应的C语句
- 能用GDB调试程序
- 养成细致严谨的态度，注意C语言与汇编语言之间的关系，培养系统观

算术运算类指令课堂练习（学习通）



下面指令执行后eax中的值是什么？用公式表示

```
movsx eax, cx
```

```
imul eax,9
```

```
cdq
```

```
mov ebx,5
```

```
idiv ebx
```

```
add eax,32
```



位操作类指令

位操作类指令

- 计算机中最基本的数据单位是二进制位
- 针对二进制位进行操作、实现位控制的指令
 - 逻辑运算指令
 - 移位指令
 - 循环移位指令
 -
- 进行一位或若干位处理，采用位操作类指令

多多阅读程序

AND、TEST、OR、XOR和NOT指令	逻辑运算
SHL/SAL、SHR和SAR指令	移位
ROL、ROR、RCL和RCR指令	循环移位

设置CF = OF = 0
影响SF, ZF和PF

逻辑运算类指令

AND reg,imm/reg/mem	$\#reg \leftarrow reg \wedge imm/reg/mem$
AND mem,imm/reg	$\#mem \leftarrow mem \wedge imm/reg$
TEST reg,imm/reg/mem	$\#reg \wedge imm/reg/mem$
TEST mem,imm/reg	$\#mem \wedge imm/reg$
OR reg,imm/reg/mem	$\#reg \leftarrow reg \vee imm/reg/mem$
OR mem,imm/reg	$\#mem \leftarrow mem \vee imm/reg$
XOR reg,imm/reg/mem	$\#reg \leftarrow reg \oplus imm/reg/mem$
XOR mem,imm/reg	$\#mem \leftarrow mem \oplus imm/reg$
NOT reg/mem	$\#reg/mem \leftarrow \sim reg/mem$

不影响状态标志位

逻辑运算指令有什么作用?

逻辑运算的屏蔽作用

- **AND**复位某些位(同0与), 不影响其他位(同1与)
- **OR**置位某些位(同1或), 不影响其他位(同0或)
- **XOR**求反某些位(同1异或), 不影响其他位(同0异或)

`and bl,0b11110110` #BL中D0和D3清0, 其余位不变

`or bl,0b00001001` #BL中D0和D3置1, 其余位不变

`xor bl,0b00001001` #BL中D0和D3求反, 其余位不变

 置位**Set**: 置1

 复位**Reset**: 清0, 清除**Clear**

大写=小写 **AND 0xDF**
小写=大写 **OR 0x20**

```
xor edx, edx
sub edx, edx
mov edx, 0
```


移位指令



- 逻辑左移指令 SHL
- 逻辑右移指令 SHR
- 算术左移指令 SAL
- 算术右移指令 SAR
- 双精度左移指令 SHLD
- 双精度右移指令 SHRD

- 不带进位循环左移指令 ROL
- 不带进位循环右移指令 ROR
- 带进位循环左移指令 RCL
- 带进位循环右移指令 RCR

S: Shift

R: Rotate

L: Left

R: Right

A: Arithmetic

C: Carry flag

D: 双精度

1. 移位指令

- 分逻辑 (Logical) 和算术 (Arithmetic) 移位
- 具有左移 (Left) 或右移 (Right) 操作

SHL reg/mem,i8/CL

#逻辑左移：最低位补0，最高位进入CF

SHR reg/mem,i8/CL

#逻辑右移：最高位补0，最低位进入CF

SAL reg/mem,i8/CL

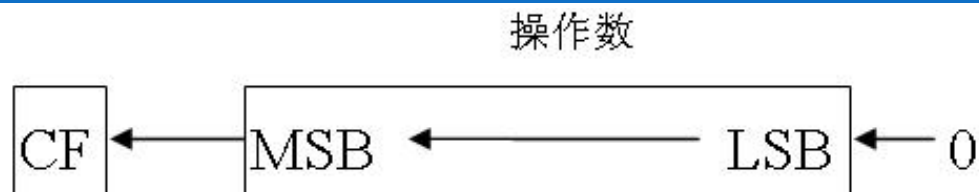
#算术左移，与SHL是同一条指令

SAR reg/mem,i8/CL

#算术右移：最高位不变，最低位进入CF

- 目的操作数：寄存器或存储单元reg/mem
- 后一个操作数：移位位数i8/CL

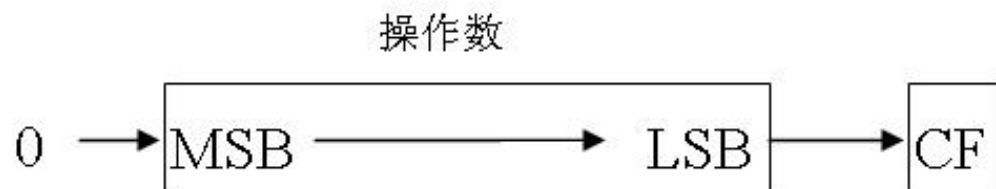
移位指令的功能和示例



(a) 逻辑左移 SHL/算术左移 SAL

执行前
执行的指令
执行后

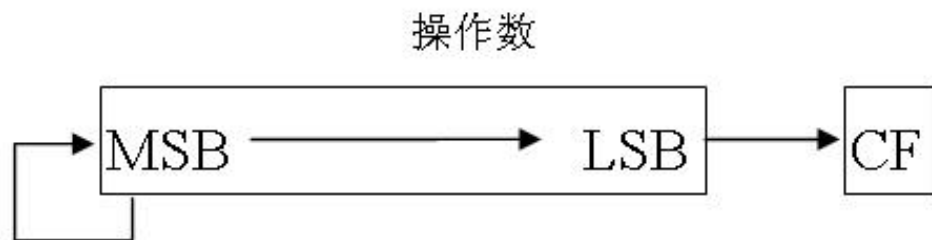
AL = 11110101B
SHL AL, 1
AL = 11101010B
CF = 1, OF = 0



(b) 逻辑右移 SHR

执行前
执行的指令
执行后

AL = 11110101B
SHR AL, 1
AL = 01111010B
CF = 1, OF = 1



(c) 算术右移 SAR

执行前
执行的指令
执行后

AL = 11110101B
SAR AL, 1
AL = 11111010B
CF = 1, OF = 0

用移位指令实现算术运算



- 逻辑左移一位，相当于无符号数乘以2，逻辑左移 n 位，相当于无符号数乘以 2^n ；
- 逻辑右移一位，相当于无符号数除以2，逻辑右移 n 位，相当于无符号数除以 2^n ；
- 商在目的操作数中，余数由CF标志反映。
- 算术右移一位，相当于有符号数除以2，算术右移 n 位，相当于有符号数除以 2^n 。
- 当操作数为负数，并且最低位有1移出时，算术右移与等效的IDIV指令的结果不同。

例如：EDX $\leftarrow$ 3 $\times$ EAX + 7 $\times$ EBX

分析：3 $\times$ EAX + 7 $\times$ EBX = 2 $\times$ EAX + 1 $\times$ EAX + 8 $\times$ EBX - 1 $\times$ EBX

```
mov esi,eax
shl esi,1      #esi $\leftarrow$ 2 $\times$ eax
add esi,eax    #esi $\leftarrow$ 3 $\times$ eax
mov edx,ebx
shl edx,3      #edx $\leftarrow$ 8 $\times$ ebx
sub edx,ebx    #edx $\leftarrow$ 7 $\times$ ebx
add edx,esi    #edx $\leftarrow$ 7 $\times$ ebx + 3 $\times$ eax
```

下面指令实现了什么功能?



```
xor eax,eax  
mov ax,wvar  
shl eax,1  
mov ebx,eax  
shl eax,2  
add eax,ebx  
call dispuid  
call dispCrLf  
imul eax,10  
call dispuid
```

```
wvar: .word 123
```

 **注意：配套子程序有可能改变RAX的值**

 **SHL**逻辑左移一位相当于无符号数乘以2

 **SHR**逻辑右移一位相当于无符号数除以2

C语言的右移操作



- C语言标准没有明确定义应该使用哪种右移操作

- 对于无符号数
必须使用逻辑右移

操作	数值（二进制）	
<code>unsigned char x;</code>	<code>01100110</code>	<code>10010111</code>
<code>x >> 1</code>	<code>00110011</code>	<code>01001011</code>
<code>x >> 4</code>	<code>00000110</code>	<code>00001001</code>

- 对于有符号数
几乎所有编译器
都采用算术右移

操作	数值（二进制）	
<code>char x;</code>	<code>01100110</code>	<code>10010111</code>
<code>x >> 1</code>	<code>00110011</code>	<code>11001011</code>
<code>x >> 4</code>	<code>00000110</code>	<code>11111001</code>

循环移位指令

- 循环 (Rotate) 移位指令要将从一端移出的位返回到另一端形成循环
- 分成不带进位循环移位和带进位循环移位
- 分别具有左移或右移操作

ROL reg/mem,i8/CL #不带进位循环左移指令

ROR reg/mem,i8/CL #不带进位循环右移指令

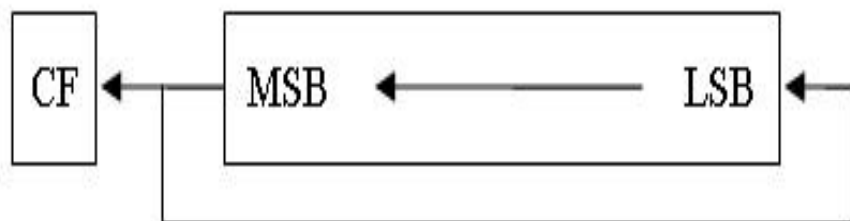
RCL reg/mem,i8/CL #带进位循环左移指令

RCR reg/mem,i8/CL #带进位循环右移指令

不带进位的循环移位指令



操作数

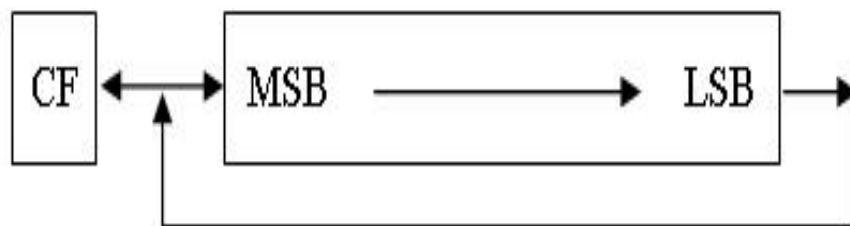


(a) 左循环移位 ROL

执行前
执行的指令
执行后

AL=11100101B
ROL AL,1
AL=11001011B
CF=1, OF=0

操作数



(b) 右循环移位 ROR

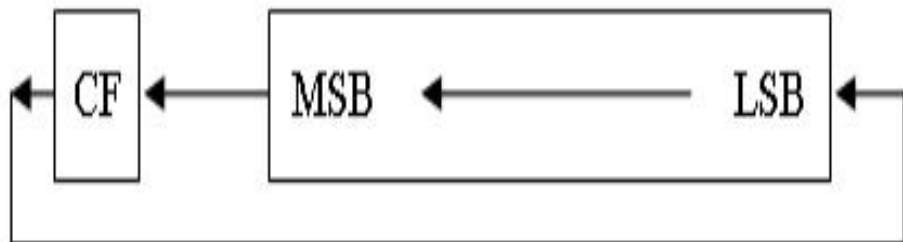
执行前
执行的指令
执行后

AL=11100101B, CL=2
ROR AL,CL
AL=01111001B, CL=2
CF=0

带进位的循环移位指令



操作数



(c) 带进位左循环移位 RCL

执行前
执行的指令
执行后

AL=11100101B, CF=0

RCL AL,1

AL=11001010B

CF=1, OF=0

操作数



(d) 带进位右循环移位RCR

执行前
执行的指令
执行后

AL=11100101B, CF=0

RCR AL,4

AL=10101110B

CF=0

课堂练习



1. 指令 “shl eax, 1” 执行后, EAX的最低位一定是:

A 0

B 1

C 与最高位相同

D 与进位CF标志相同

答案: A

2. 指令 “ror eax, 1” 执行后, EAX的最高位一定是:

A 0

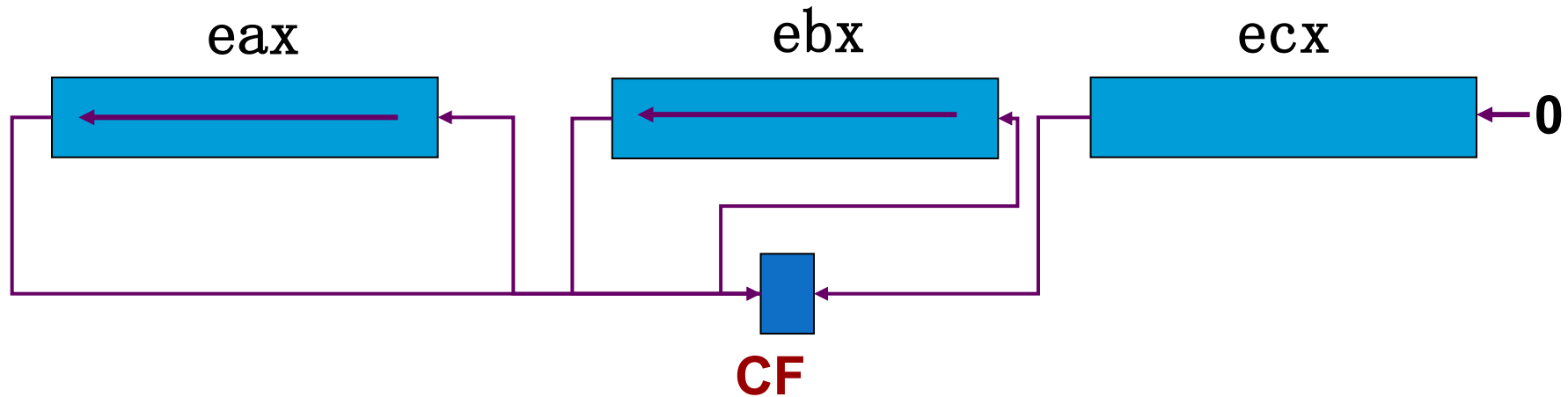
B 1

C 与最低位相同

D 与进位CF标志相同

答案: D

96位数据左移1位

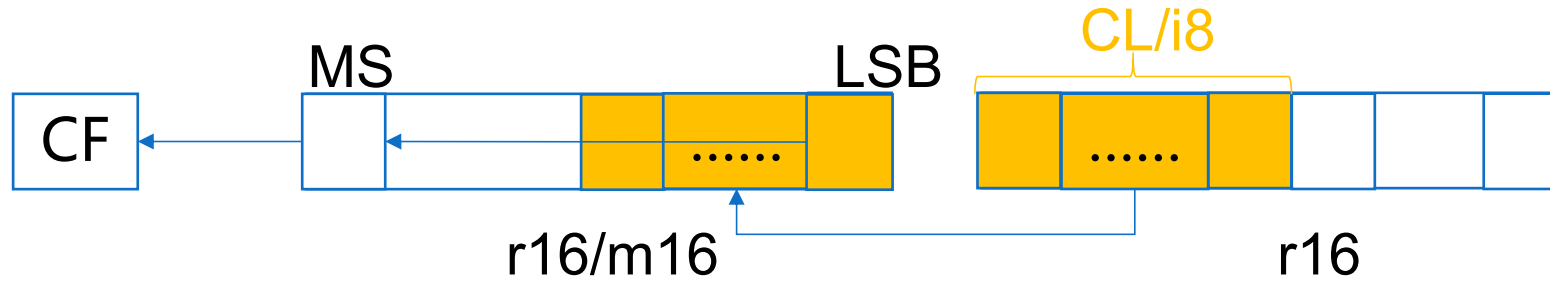


`shl ecx,1` #ecx的最高位移入CF

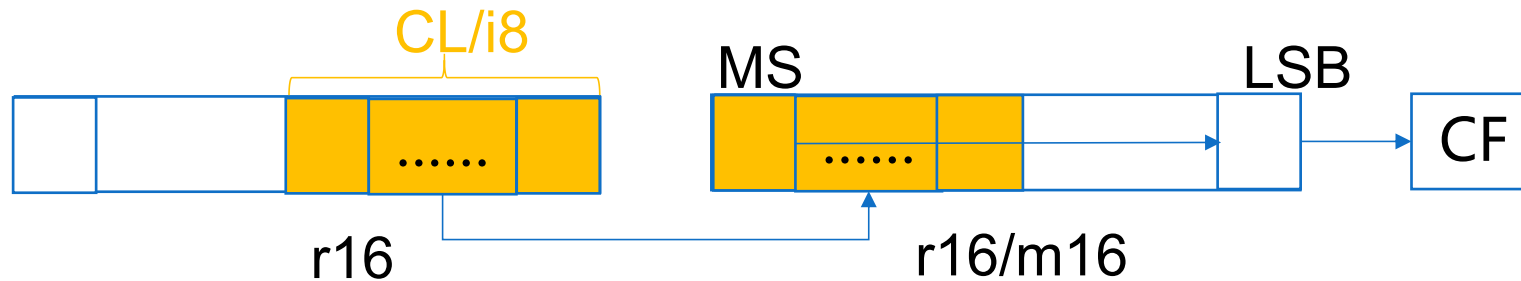
`rcl ebx,1` #CF移入ebx的最低位, ebx的最高位移入CF

`rcl eax,1` #CF移入eax最低位

双精度移位指令



SHLD r16/m16, r16, CL/i8
SHLD r32/m32, r32, CL/i8
SHLD r64/m64, r64, CL/i8



SHRD r16/m16, r16, CL/i8
SHRD r32/m32, r32, CL/i8
SHRD r64/m64, r64, CL/i8

双精度移位指令示例



mov ebx,0x5678

mov ecx,0x1234

mov eax,0x9abc

shld ecx,ebx,4

#ecx=0x12340, ebx=0x5678, CF=0

shrd eax,ebx,8

#eax=0x7800009a, ebx=0x5678, CF=1

mov eax,0x12345678

#eax=0x12345678

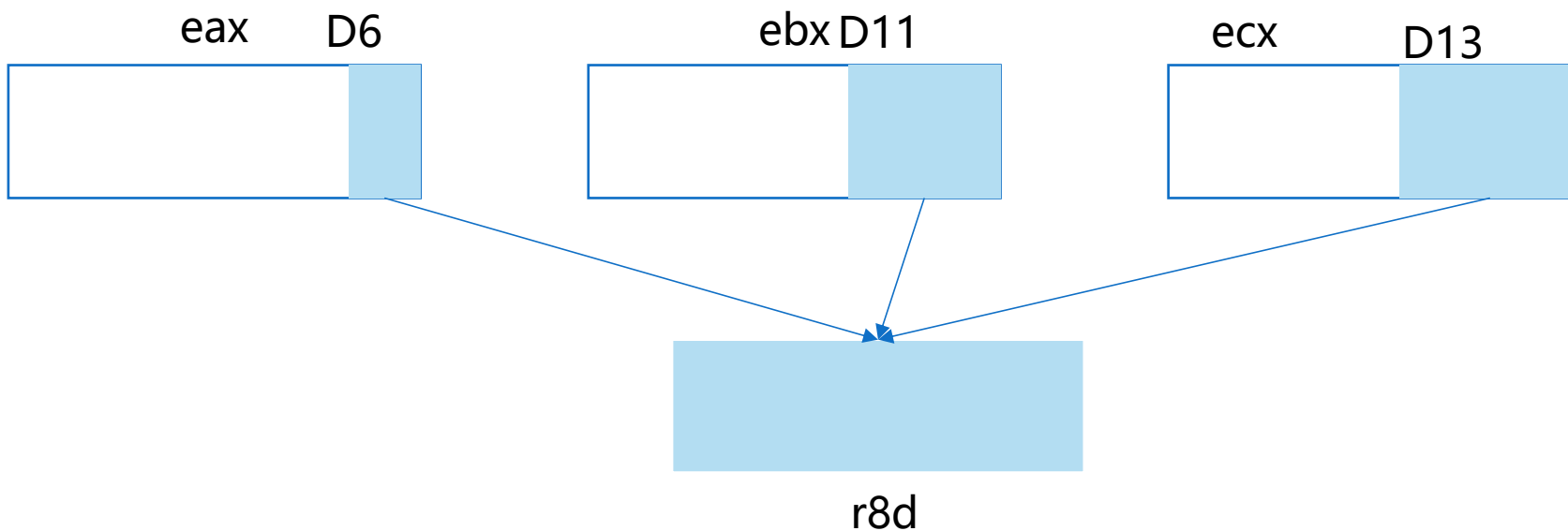
mov ebx,0xffffffff

#ebx=0xffffffff

shld eax,ebx,4

#eax=0x2345678f, ebx=0xffffffff, eflags=0x203

将寄存器eax低7位,ebx低12位和ecx低13位组合成一个新的双字保存到r8d寄存器中。



```
shrd r8d, ecx, 13  
shrd r8d, ebx, 12  
shrd r8d, eax, 7
```

如何实现shld ebx,eax,1的功能

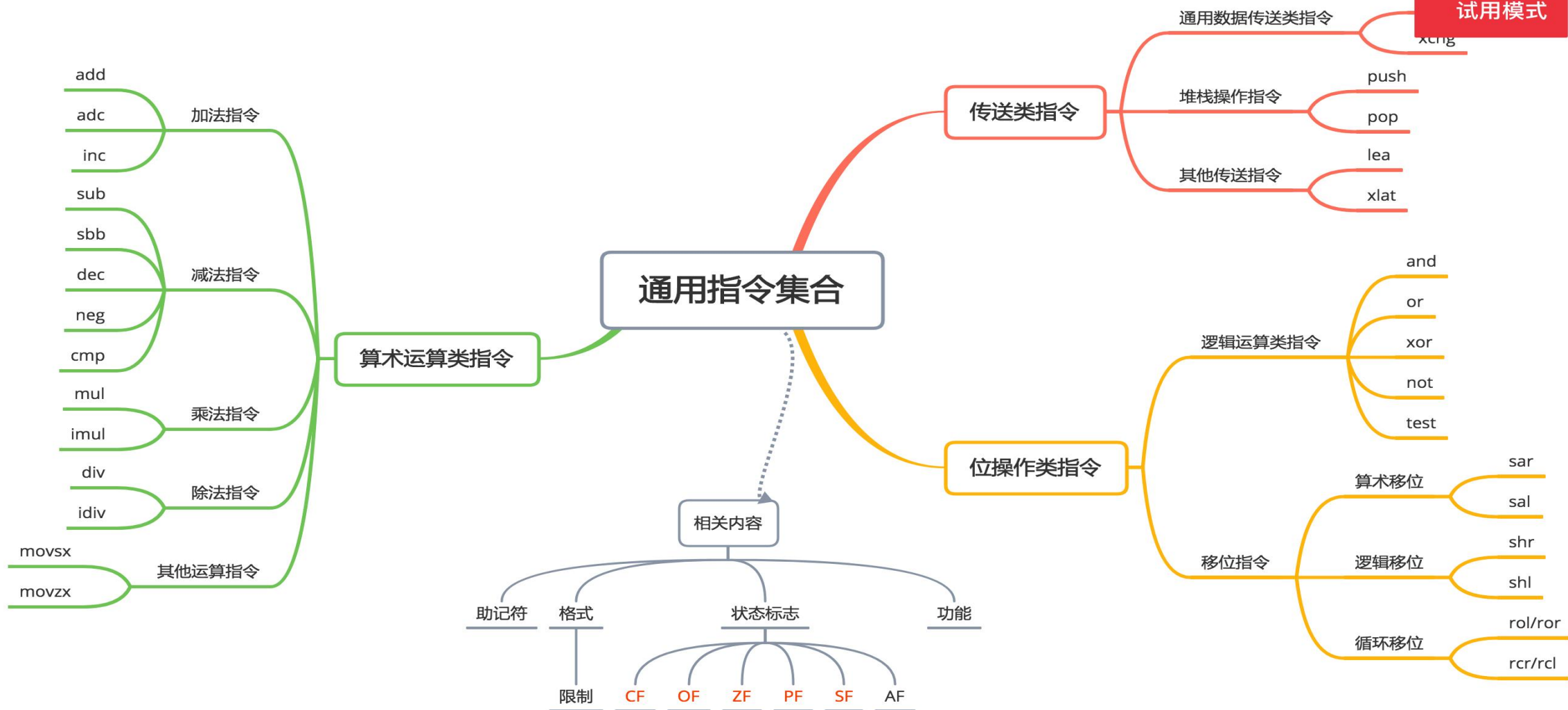


```
rol  eax,1  #eax的最高位移入CF  
rcl  ebx,1  #CF移入ebx的最低位  
ror  eax,1  #恢复eax
```

传送类、算术运算类和移位指令总结



XMind
试用模式



讨论：传送类指令和算术运算类指令应用



任务描述

编写一个能进行混合四则运算的汇编语言程序。

编程要求

在寄存器EAX, EBX, ECX, EDX中分别存放了四个变量X,Y,V,Z的值，均为 32 位有符号数。此外，还有一个寄存器RSI保存有变量W的地址。

请编写汇编代码，计算 $W=(X*Y-V)/Z$ 的结果，并将商保存到W位置处，将余数保存到W+4位置处。

注意：算式中的乘法运算不会超出32位表示范围，减法运算不会得到负数结果。

讨论：用C语言完成上述功能，用编译软件采用不同的优化参数生成汇编代码，你的代码与编译器生成的代码哪一个更优？

【例4-15】计算 $1 + 2 + 3 + \dots + N$ 的和

- 利用等差数列求和公式

$$1 + 2 + 3 + \dots + N \\ = (1 + N) \times N \div 2$$

获得N值

N+1

$(1 + N) \times N$

结果除以2

保存结果

```
unsigned long sum(unsigned long N){  
    return (1+N)*N/2;  
}
```

```
.data  
N:    .long 3456 #假设一个N值 (小于 $2^{32}-1$ )  
.code  
mov ecx, DWORD PTR N[rip] # ECX=N  
lea ecx, 1[rcx]           # add ecx, 1  
imul eax, ecx             # EAX=(1+N)×N  
shr eax, 1                # EAX=(1+N)×N÷2
```

扩大精度



```
.data
N:      .long    3456    #N小于232-1
.code
mov ecx, DWORD PTR N[rip] # ECX=N
lea ecx, 1[rcx]           # add ecx, 1
imul eax, ecx             # EAX=(1+N)×N
shr eax,1                 # EAX=(1+N)×N÷2
```

```
.data
N      .long 3456
sum    .quad 0
.code
mov eax,N[rip]           #EAX=N
add eax,1                #EAX=N+1
mul dword ptr N[rip]     #EDX.EAX=(1+N)×N
shr edx,1                #64位逻辑右移一位(除以2)
rcr eax,1                #EDX.EAX= EDX.EAX÷2
mov dword ptr [sum],eax  #小端方式保存
mov dword ptr [sum+4],edx
```

〔例4-16〕 温度转换程序



摄氏温度C转换为华氏温度F的公式：

$$F = (9/5) \times C + 32$$

```
short CtoF(short C){  
    short F;  
    F=9*C/5+32;  
    return F;  
}
```

```
movsxcx, cx  
# 摄氏温度C保存在cx寄存器中，符号扩展到ecx中  
lea     edx, [rcx+rcx*8]      # 9×C  
movsxxax, edx                # rax = 9×C  
imul    rax, rax, 1717986919  
sar     rax, 33  
sar     edx, 31  
sub     eax, edx              # 9×C/5  
add     eax, 32               # 9×C/5+ 32
```

gcc -S -Og -masm=intel -o ./progs/eg0421.S ./progs/eg0421.c

除法优化



```
movsxcx, cx
lea     edx, [rcx+rcx*8]    # 9×C
movsxrax, edx               # rax = 9×C
imul    rax, rax, 1717986919
sar     rax, 33
sar     edx, 31
sub     eax, edx            # 9×C/5
add     eax, 32             # 9×C/5+ 32
```

```
movsx eax, cx
imul eax, 9
cdq
mov ebx, 5
idiv ebx    # EAX = C×9/5 (没有考虑余数)
add eax, 32 # EAX = F = C×9/5 + 32
```

〔例4-17〕 C语言的算术运算



```
long arith(long x, long y, long z)
{
    long t1 = x ^ y;
    long t2 = z * 48;
    long t3 = t1 & 0x0F0F0F0F;
    long t4 = t2 - t3;
    return t4;
}
```

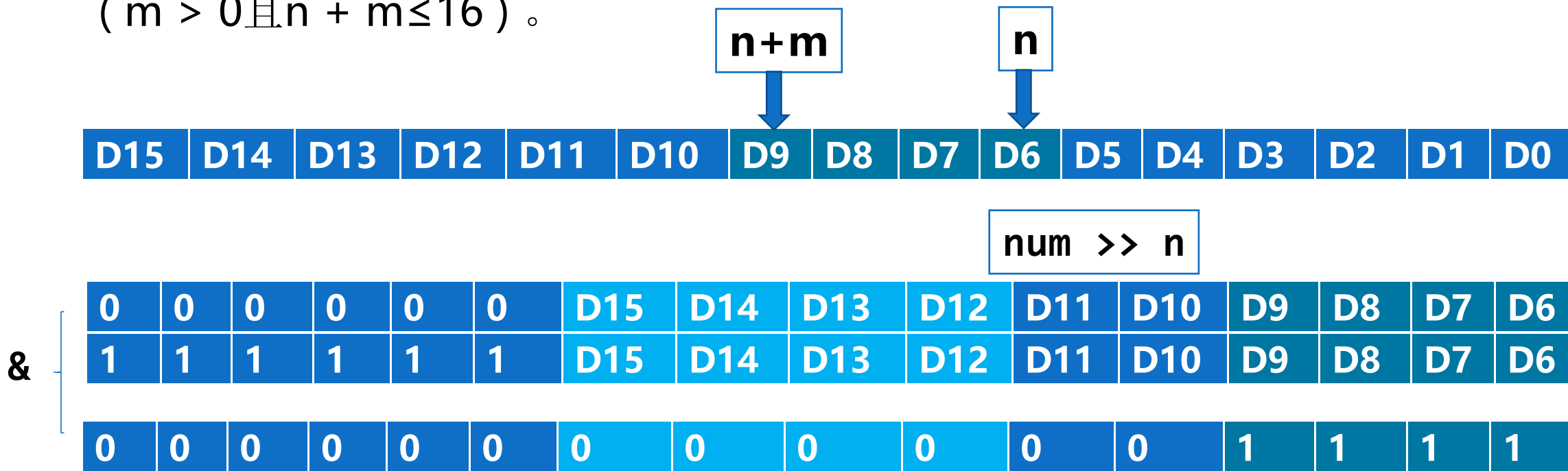
ecx:x, edx:y, r8:z

xor	ecx, edx	# t1 = x ^ y
lea	eax, [r8+r8*2]	# 3×z
sal	eax, 4	# t2 = z ×48
and	ecx, 252645135	# t3 = t1 & 0x0F0F0F0F;
sub	eax, ecx	# t4 = t2 - t3;

〔例4-18〕 截取m位数据



设整数占两个字节，求一个整数num，从右端第n($0 \leq n \leq 15$)位开始的m位
($m > 0$ 且 $n + m \leq 16$)。



C语言表达式: $(\text{num} \gg n) \& (\sim (\sim 0 \ll m))$

$\sim (\sim 0 \ll m)$

$(\text{num} \gg n) \& (\sim (\sim 0 \ll m))$



```
movsx    edx, WORD PTR num[rip]
movzx    ecx, BYTE PTR n[rip]
sar      edx, cl          # num >> n
movzx    ecx, BYTE PTR m[rip]
mov      eax, -1
sal      eax, cl          # ~0 << m
not      eax              # ~ ( ~0 << m )
and      edx, eax         #(num >> n) & ( ~ ( ~0 << m ) )
```